```julia
begin
    using LinearAlgebra
    using PlutoUI
    using PlutoTeachingTools
    using Plots
    using LaTeXStrings
    using HypertextLiteral: @htl, @htl_str
end
```

Table of Contents

Eigenvalue problems

Power iteration

⚠ TODO ⚠

⚠ TODO ⚠

⚠ TODO ⚠

Power iteration algorithm

Convergence of power method

Spectral transformations

⚠ TODO ⚠

Optional: Dynamic shifting

Eigenvalue problems

In the previous notebooks we already noted a connection between the eigenspectrum of the iteration matrix and the convergence properties of fixed-point problems or the convergence properties of solving linear systems.

In this notebook we will discuss some simple iterative methods for actually computing eigenpairs. However, the topic is vast and we will only scratch the surface. Readers interested in a more in-depth treatment of eigenvalue problems are encouraged to attend the master class MATH-500: Error control in scientific modelling.

A more comprehensive treatment of the topic can also be found in the book Numerical Methods for Large Eigenvalue Problems by Youssef Saad as well as the Lecture notes on Large Scale Eigenvalue Problems by Peter Arbenz.

Power iteration

We start with a simple question: What happens if we apply a matrix multiple times ? Here, we choose the matrix

```
A = 2×2 Matrix{Float64}:
     0.5  0.333333
     0.5  0.666667
```

```
A = [ 0.5  1/3;
      0.5  2/3]
```

and a random 2-element vector:

```
[-0.660899, -0.313369]
```

```
begin
    dummy  # For rerun to work
    x = randn(2)
end
```

```
[-0.660899, -0.313369]
```

```
x
```

Applying the matrix once seems to be very innocent:

```
[-0.434906, -0.539362]
```

```julia
A * x
```

But if we appy many times we start to see something ...

```
2×2 Matrix{Float64}:
 -0.389713  -0.389708
 -0.584555  -0.58456
```

```julia
begin
    for j in 1:6
        x = A * x
    end
    maxerror = maximum(abs, (x - A * x))
    @show maxerror
    [x A * x]
end
```

```
maxerror = 4.843822741684711e-6
```

Regenerate random matrix and rerun experiment

⚠ TODO ⚠

Graphical visualisation

```
TODO("Graphical visualisation")
```

Note how the iterations stabilise, i.e. that $\mathbf{x}$ and $\mathbf{Ax}$ start to be alike. In other words we seem to achieve $\mathbf{Ax} = \mathbf{x}$, which is nothing else than saying that $\mathbf{x}$ is an eigenvector of $\mathbf{A}$ with eigenvalue 1.

⚠ TODO ⚠

Warning box about eigenvalue ordering: Note explicitly the order of Julia and the orderings we use here. Note that we use different eigenvalue orderings depending on the algorithm which is discussed and the context that one focuses on. Note in partiuclar that λ_1 and λ_2 throughout these notes may mean different things !

Note, that in this developent we have cheated a little, namely we allowed ourself to normalise the columns (the `M = M ./ sum(M; dims=1)` above).

Let us understand what happened in this example in detail now. We consider the case $\mathbf{A} \in \mathbb{R}^{n \times n}$ diagonalisable and let further

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n| \tag{1}$$

be its eigenvalues with corresponding eigenvectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n$, which we collect column-wise in a unitary matrix $\mathbf{V}$, i.e.

$$\mathbf{V} = \begin{pmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_n \\ \downarrow & \downarrow & \cdots & \downarrow \end{pmatrix}$$

Note that $|\lambda_1| > |\lambda_2|$, such that λ_1 is non-degenerate and the absolutely largest eigenvalue. We call it the **dominant eigenvalue** of $\mathbf{A}$. If $k > 0$ is a positive integer, then we have

$$\mathbf{A}^k = \left(\mathbf{V}\,\mathbf{D}\,\mathbf{V}^{-1}\right)^k = \mathbf{V}\,\mathbf{D}^k\,\mathbf{V}^{-1}$$

and we can expand any random starting vector $\mathbf{x}^{(1)}$ in terms of the eigenbasis of $\mathbf{A}$, i.e.

$$\mathbf{x}^{(1)} = \sum_{i=1}^{n} z_i \mathbf{v}_i = \mathbf{V}\mathbf{z},$$

where $\mathbf{z}$ is the vector collecting the coefficients z_i. Notably this implies $\mathbf{z} = \mathbf{V}^{-1}\mathbf{x}^{(1)}$.

We now consider applying $\mathbf{A}$ a k number of times to $\mathbf{x}^{(1)}$, which yields

$$
\begin{aligned}
\mathbf{A}^k \mathbf{x}^{(1)} &= \mathbf{V}\,\mathbf{D}^k \underbrace{\mathbf{V}^{-1}\mathbf{x}^{(1)}}_{=\mathbf{z}} = \mathbf{V} \begin{pmatrix} \lambda_1^k z_1 \\ \lambda_2^k z_2 \\ \vdots \\ \lambda_n^k z_n \end{pmatrix} \\
&= \lambda_1^k \left(z_1 \mathbf{v}_1 + \left(\frac{\lambda_2}{\lambda_1}\right)^k z_2 \mathbf{v}_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^k z_n \mathbf{v}_n \right).
\end{aligned} \tag{2}
$$

Therefore if $z_1 \neq 0$ then

$$
\left\| \frac{\mathbf{A}^k \mathbf{x}^{(1)}}{\lambda_1^k} - z_1 \mathbf{v}_1 \right\| \leq \left| \frac{\lambda_2}{\lambda_1} \right|^k |z_2| \, \|\mathbf{v}_2\| + \cdots + \left| \frac{\lambda_n}{\lambda_1} \right|^k |z_n| \, \|\mathbf{v}_n\| \tag{3}
$$

Now, each of the terms $\left|\frac{\lambda_j}{\lambda_1}\right|^k$ for $j = 2, \ldots, n$ goes to zero for $k \to \infty$ due to the descending eigenvalue ordering of Equation (1). Therefore overall

$$
\left\| \frac{\mathbf{A}^k \mathbf{x}^{(1)}}{\lambda_1^k} - z_1 \mathbf{v}_1 \right\| \to 0 \qquad \text{as } k \to \infty.
$$

In other words $\frac{1}{\lambda_1^k} \mathbf{A}^k \mathbf{x}^{(1)}$ eventually becomes a multiple of the eigenvector associated with the dominant eigenvalue.

⚠ TODO ⚠

First show that generalising to a 2x2 matrix without the rescaling trick fails because the entries in the iterated vector $\boldsymbol{x}$ get larger and larger.

Then suggests to rescale the entries somehow, one natural idea is to simply use the largest by magnitude entry. Show that this fixes the issue.

Then motivate why β is a good estimate for the eigenvalue, show the algorithm, show its convergence and then show its convergence proof.

Power iteration algorithm

In order to turn this idea into an algorithm there are two aspects missing:

- λ_1^k becomes either extremely small (for $\lambda_1 < 1$) or extremely large (for $\lambda_1 > 1$) if $k \to \infty$. As a result representing the matrix entries of $\mathbf{A}^k\mathbf{x}$ and performing the associated matrix-vector products numerically accurately in finite-precision floating-point arithmetic is difficult. Some form of *normalisation* at each step of our iteration techinque is thus required.
- For the normalisation we cannot simply divide by λ_1 itself, since this eigenvalue is unknown to us.

But other choices of normalisation exist. In this case we will take the infinity norm:

$$\|x\|_\infty = \max_{i=1,\dots n} |x_i|,$$

which simply selects the largest absolute entry of the vector. Based on this idea we obtain the algorithm

Algorithm 1: Power iterations

Given a diagonalisable matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ and an initial guess $\mathbf{x}^{(1)} \in \mathbb{R}^n$ we iterate for $k = 1, 2, \dots$:

1. Set $\mathbf{y}^{(k)} = \mathbf{A}\,\mathbf{x}^{(k)}$
2. Find the index m such that $\left|y_m^{(k)}\right|$ is the largest (by magnitude) element of $\mathbf{y}^{(k)}$, i.e.

$$m = \mathbf{argmax}_i \left| y_i^{(k)} \right|$$

3. Set $\alpha^{(k)} = \frac{1}{y_m^{(k)}}$ and $\beta^{(k)} = \frac{y_m^{(k)}}{x_m^{(k)}}$.
4. Set $\mathbf{x}^{(k+1)} = \alpha^{(k)} \mathbf{y}^{(k)}$

Note for this algorithm we can write

$$\mathbf{x}^{(k)} = \alpha^{(1)} \cdot \alpha^{(2)} \cdot \ldots \cdot \alpha^{(k)} \cdot \mathbf{A}^k \mathbf{x}^{(1)} = \prod_{i=1}^{k} \alpha^{(i)} \cdot \mathbf{A}^k \mathbf{x}^{(1)} \tag{4}$$

and by construction we always have $\left\| \mathbf{x}^{(k)} \right\|_\infty = 1$. Algorithm 1 thus only presents a minor modification over the scheme we discussed in the previous section.

Finally we note that if $\mathbf{x}^{(k)}$ is almost an eigenvector of the dominant eigenvalue, then $\mathbf{A}\,\mathbf{x}^{(k)}$ is almost $\lambda_1 \mathbf{x}^{(k)}$ and thus one would assume $\beta^{(k)} = \frac{y_m^{(k)}}{x_m^{(k)}}$ to be a reasonable eigenvalue estimate. Indeed note that

$$\begin{aligned}
\beta^{(k)} &= \frac{y_m^{(k)}}{x_m^{(k)}} = \frac{\left(\mathbf{A}\,\mathbf{x}^{(k)}\right)_m}{x_m^{(k)}} \\
&\overset{(4)}{=} \frac{\left(\prod_{i=1}^{k} \alpha^{(i)} \cdot \mathbf{A}^{k+1} \mathbf{x}^{(1)}\right)_m}{\left(\prod_{i=1}^{k} \alpha^{(i)} \cdot \mathbf{A}^{k} \mathbf{x}^{(1)}\right)_m} \\
&\overset{(2)}{=} \frac{\lambda_1^{k+1} \left(z_1 \mathbf{v}_1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k+1} z_2 \mathbf{v}_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k+1} z_n \mathbf{v}_n \right)_m}{\lambda_1^{k} \left(z_1 \mathbf{v}_1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k} z_2 \mathbf{v}_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k} z_n \mathbf{v}_n \right)_m} \\
&= \frac{\lambda_1 \left(b_1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k+1} b_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k+1} b_n \right)}{b_1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k} b_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k} b_n}
\end{aligned}$$

where $_m$ denotes that we take the m-th element of the vector in the brackets and we further defined $b_i = z_i (\mathbf{v}_i)_m$, that is z_i multiplied by the m-th element of the eigenvector $\mathbf{v}_i$. Again we assume $z_1 \neq 0$, which implies $b_1 \neq 0$. Under this assumption

$$\beta^{(k)} = \frac{y_m^{(k)}}{x_m^{(k)}} = \lambda_1 \frac{1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k+1} \frac{b_2}{b_1} + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k+1} \frac{b_n}{b_1}}{1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k} \frac{b_2}{b_1} + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k} \frac{b_n}{b_1}} \to \lambda_1 \qquad \text{as } k \to \infty. \tag{5}$$

Keeping in mind that $\frac{\lambda_i}{\lambda_1} < 1$ for all $2 \leq i \leq n$ we indeed observe $\beta^{(k)}$ to converge to λ_1.

An implementation of this power method algorithm in:

```
power_method (generic function with 1 method)
```

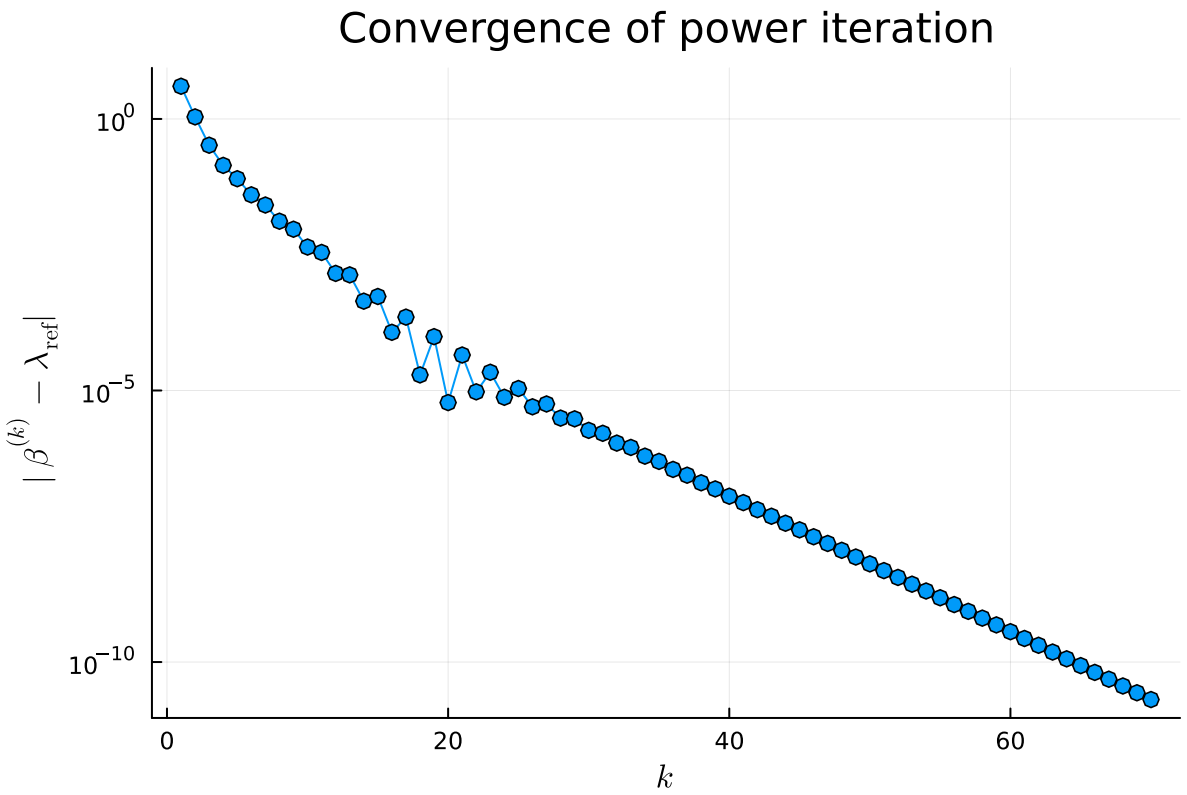
```julia
function power_method(A, x; maxiter=100)
    n = size(A, 1)
    x = normalize(x, Inf)  # Normalise initial guess

    history = Float64[]  # Record a history of all βs (estimates of eigenvalue)
    for k in 1:maxiter
        y = A * x
        m = argmax(abs.(y))
        α = 1 / y[m]
        β = y[m] / x[m]
        push!(history, β)
        x = α * y
    end

    (; x, λ=last(history), history)
end
```

Let's try this on a lower-triangular matrix as a test. Note that the eigenvalues of a lower-triangular matrix are exactly the diagonal entries. We set

```
5×5 Matrix{Float64}:
 1.0   1.0   1.0   1.0  1.0
 0.0  -0.75  1.0   1.0  1.0
 0.0   0.0   0.6   1.0  1.0
 0.0   0.0   0.0  -0.4  1.0
 0.0   0.0   0.0   0.0  0.0
```

```julia
begin
    λref = [1, -0.75, 0.6, -0.4, 0]
    # Make a triangular matrix with eigenvalues on the diagonal.
    M = triu(ones(5,5),1) + diagm(λref)
end
```

By construction the largest eigenvalue of this matrix is 1. So let's track convergence:




```
begin
    λlargest = 1.0  # Largest eigenvalue of M (by construction)

    x1 = ones(size(M, 2))
    results = power_method(M, x1; maxiter=70)
    error = abs.(results.history .- λlargest)
    plot(error; mark=:o, label="", yaxis=:log,
         title="Convergence of power iteration",
         ylabel=L"|β^{(k)} - λ_{\textrm{ref}}|", xlabel=L"k")
end
```

Remark on $z_1 \neq 0$

In the theoretical discussions so far we always assumed $z_1 \neq 0$, i.e. that the initial guess $\mathbf{x}^{(1)}$ has a non-zero component along the eigenvector $\mathbf{v}_1$ of the dominant eigenvalue. This may not always be the case. However even if $z_1 \neq 0$, computing $\mathbf{A}\mathbf{x}^{(1)}$ in finite-precision floating-point arithmetic is associated with a small error, which means that which implies that $\mathbf{A}\mathbf{x}^{(1)}$ and thus $\mathbf{x}^{(2)}$ does usually *not* have a zero component along $\mathbf{v}_1$, such that in practical calculations assuming $z_1 \neq 0$ is not a restriction.

Convergence of power method

The above plot already suggests a **linear convergence** towards the exact eigenvalue. Indeed a detailed analysis shows that the convergence rate can be computed as

$$r_{\text{power}} = \lim_{k\to\infty} \frac{|\beta^{(k+1)} - \lambda_1|}{|\beta^{(k)} - \lambda_1|} = \left|\frac{\lambda_2}{\lambda_1}\right|. \tag{6}$$

▶ **Detailed derivation**

So the smaller the ratio between $|\lambda_2|$ and $|\lambda_1|$, the faster the convergence.

Let us take a case where $\lambda_1 = 1.0$. Therefore the size of λ_2 determines the rate of convergence. Let's take λ_2 = -0.9.

```
▶ [1.0, -0.9, 0.6, -0.4, 0.0]
```

```
begin
    λ₁ = 1
    λ_δ = [λ₁, λ₂, 0.6, -0.4, 0]   # The reference eigenvalues we will use
end
```

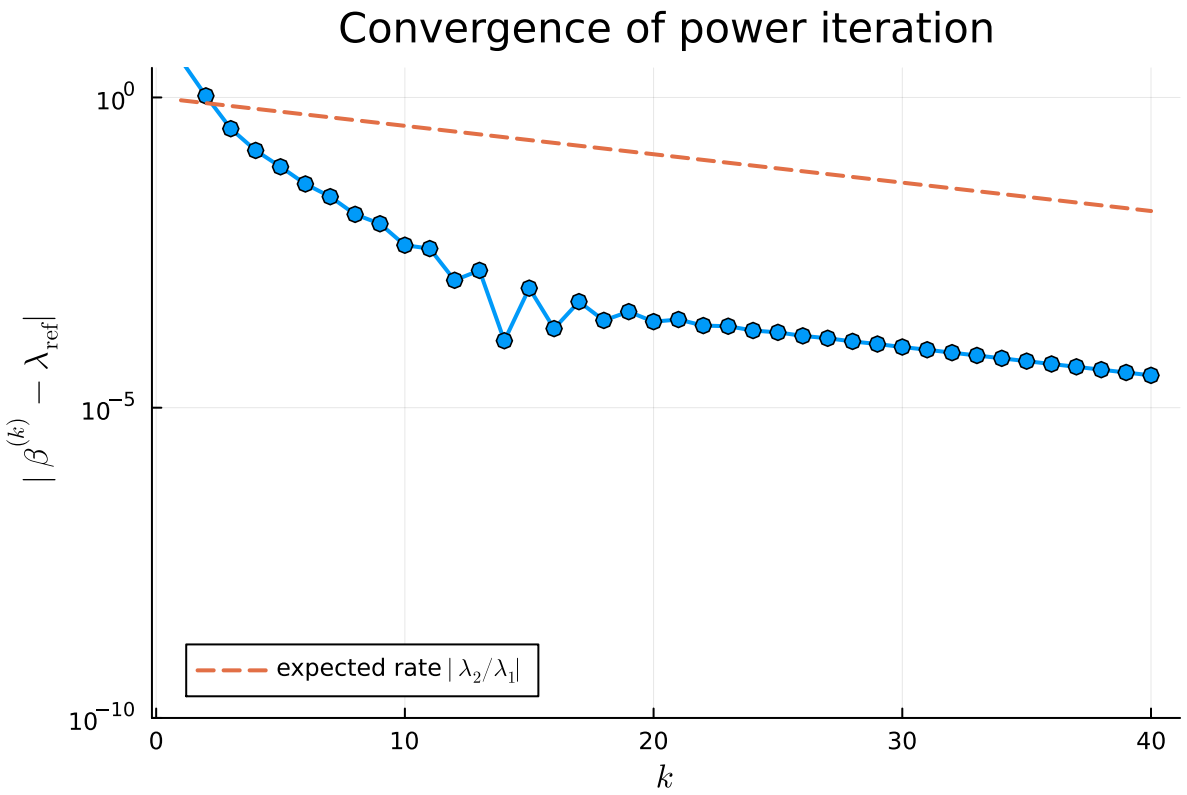
and we will put these on the diagonal of a triangular matrix, such that the eigenvalues of the resulting matrix M are exactly the λ_δ:

```
M_δ = 5×5 Matrix{Float64}:
      1.0   1.0  1.0   1.0  1.0
      0.0  -0.9  1.0   1.0  1.0
      0.0   0.0  0.6   1.0  1.0
      0.0   0.0  0.0  -0.4  1.0
      0.0   0.0  0.0   0.0  0.0
```

```
M_δ = triu(ones(5, 5), 1) + diagm(λ_δ)
```

We check this: We introduce a slider to tune the λ_2 of equation (6):

- λ_2 = -0.9




```
let
    λlargest = maximum(abs.(λref))  # Largest eigenvalue of M_δ (by construction)

    x1 = ones(size(M_δ, 2))
    results = power_method(M_δ, x1; maxiter=40)
    error = abs.(results.history .- λlargest)
    p = plot(error; mark=:o, label="", yaxis=:log, ylims=(1e-10, 3),
         title="Convergence of power iteration",
         ylabel=L"|β^{(k)} - λ_{\textrm{ref}}|",
         xlabel=L"k", legend=:bottomleft, lw=2)

    r_power = abs(λ₂ / λ₁)
    plot!(p, k -> r_power^k; ls=:dash, label=L"expected rate $| λ_2 / λ_1|$",
    lw=2)
end
```

Spectral transformations

The above procedure provides us with a simple algorithm to compute the *largest* eigenvalue of a matrix and its associated eigenvector. A natural question is now how one could compute the smallest value or some eigenvalue in between. In this section we discuss an extension to power iteration, which makes this feasible. We only need a small ingredient from linear algebra that makes this feasible, the so-called spectral transformations.

We explore based on a few examples. Consider

```
Ashift = 3×3 Matrix{Float64}:
          0.4  -0.6   0.2
         -0.3   0.7  -0.4
         -0.1  -0.4   0.5
```

```
Ashift = [ 0.4 -0.6 0.2;
           -0.3 0.7 -0.4;
           -0.1 -0.4 0.5]
```

Its eigenvalues and eigenvectors are:

```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
3-element Vector{Float64}:
 2.220446049250313e-16
 0.43944487245360087
 1.160555127546399
vectors:
3×3 Matrix{Float64}:
 0.57735   0.676008   0.636852
 0.57735  -0.272642  -0.698428
 0.57735  -0.684602   0.326522
```

```
eigen(Ashift)
```

Now we can add a multiple of the identity matrix, e.g.:

```
σ = 2
```

```
σ = 2  # Shift
```

```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
3-element Vector{Float64}:
 1.9999999999999991
 2.4394448724536018
 3.1605551275463983
vectors:
3×3 Matrix{Float64}:
 0.57735   0.676008   0.636852
 0.57735  -0.272642  -0.698428
 0.57735  -0.684602   0.326522
```

```
eigen(Ashift + σ * I)  # Add 2 * identity matrix
```

Notice, how the eigenvectors are the same and only the eigenvalues have been shifted by $\boldsymbol{\sigma}$.

Similarly we notice:

```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
3-element Vector{Float64}:
 0.3164001131587031
 0.40992932912404656
 0.5000000000000002
vectors:
3×3 Matrix{Float64}:
  0.636852  -0.676008  -0.57735
 -0.698428   0.272642  -0.57735
  0.326522   0.684602  -0.57735
```

```
let
    A⁻¹ = inv(Ashift + σ * I)
    eigen(A⁻¹)
end
```

Notice how this matrix still has the same eigenpairs (albeit in a different order) and simply the *inverted* eigenvalues of $\mathbf{A} + \sigma\mathbf{I}$.

```
[0.5, 0.409929, 0.3164]
```

```
1 ./ eigvals(Ashift + σ * I)
```

We now aim to formalise our observation more rigourously, recall that if λ_i is an eigenvalue of $\mathbf{A}$ with (non-zero) eigenvector $\mathbf{x}_i$ if and only if

$$\mathbf{A}\mathbf{x}_i = \lambda_i \mathbf{x}_i. \tag{7}$$

We usually refer to the pair $(\lambda_i, \mathbf{x}_i)$ as an **eigenpair** of $\mathbf{A}$.

The statement of the **spectral transformations** is:

Theorem 1: Spectral transformations

Assume $\mathbf{A} \in \mathbb{R}^{n \times n}$ is diagonalisable. Let $(\lambda_i, \mathbf{x}_i)$ be an eigenpair of $\mathbf{A}$, then

1. If A is invertible, then $(\frac{1}{\lambda_i}, x_i)$ is an eigenpair of A^{-1}.
2. For every $\sigma \in \mathbb{R}$ we have that $(\lambda_i - \sigma, x_i)$ is an eigenpair of $\mathbf{A} - \sigma\mathbf{I}$.
3. If $\mathbf{A} - \sigma\mathbf{I}$ is invertible, then $(\frac{1}{\lambda_i - \sigma}, x_i)$ is an eigenpair of $(\mathbf{A} - \sigma\mathbf{I})^{-1}$.

Proof: The result follows from a few calculations on top of (7). We proceed in order of the statements above.

1. Let $(\lambda_i, \mathbf{x}_i)$ be an arbitrary eigenpair of $\mathbf{A}$. Since $\mathbf{A}$ is invertible by assumption, $\lambda_i \neq 0$. Therefore for all eigenvalues λ_i of $\mathbf{A}$ the fraction $\frac{1}{\lambda_i}$ is meaningful and we can show:

$$\frac{1}{\lambda_i}\mathbf{x}_i = \frac{1}{\lambda_i}\mathbf{I}\mathbf{x}_i = \frac{1}{\lambda_i}\left(\mathbf{A}^{-1}\mathbf{A}\right)\mathbf{x}_i = \frac{1}{\lambda_i}\mathbf{A}^{-1}\left(\mathbf{A}\mathbf{x}_i\right) \overset{(6)}{=} \mathbf{A}^{-1}\mathbf{x}_i,$$

which indeed is the statement that $(\frac{1}{\lambda_i}, x_i)$ is an eigenpair of A^{-1}.

2. The argument is similar to 1 and we leave it as an exercise.
3. This statement follows by combining statements 1. and 2.

Exercise 1

Assume $\mathbf{A} \in \mathbb{R}^{n\times n}$ is diagonalisable. Prove statement 2. of Theorem 1, that is for all $\sigma \in \mathbb{R}$: If $(\lambda_i, \mathbf{x}_i)$ is an eigenpair of $\mathbf{A}$, then $\mathbf{x}_i$ is also an eigenvector of $\mathbf{A} - \sigma\mathbf{I}$ with eigenvalue $\lambda_i - \sigma$.

Consider point 1. of Theorem 1 and assume that $\mathbf{A}$ has a *smallest* eigenvalue

$$|\lambda_1| \geq |\lambda_2| \geq \cdots |\lambda_{n-1}| > |\lambda_n| > 0,$$

then $\mathbf{A}^{-1}$ has the eigenvalues

$$|\lambda_1^{-1}| \leq |\lambda_2^{-1}| \leq \cdots |\lambda_{n-1}^{-1}| < |\lambda_n^{-1}|,$$

thus a dominating (i.e. largest) eigenvalue $|\lambda_n^{-1}|$. Applying the `power_method` function (Algorithm 1) to $\mathbf{A}^{-1}$ we thus converge to $|\lambda_n^{-1}|$ from which we can deduce $|\lambda_n|$, the eigenvalue of $\mathbf{A}$ closest to zero.

Now consider point 3. and assume σ has been chosen such that

$$|\lambda_n - \sigma| \geq |\lambda_{n-1} - \sigma| \geq \cdots |\lambda_2 - \sigma| > |\lambda_1 - \sigma| > 0, \tag{8}$$

i.e. such that σ is closest to the eigenvalue λ_1. It follows

$$|\lambda_n - \sigma|^{-1} \leq |\lambda_{n-1} - \sigma|^{-1} \leq \cdots |\lambda_2 - \sigma|^{-1} < |\lambda_1 - \sigma|^{-1},$$

such that the `power_method` function converges to $|\lambda_1 - \sigma|^{-1}$. From this value we can deduce λ_1, since we know σ. In other words by applying Algorithm 1 to the shift-and-invert matrix $(\mathbf{A} - \sigma\mathbf{I})^{-1}$ enables us to find the eigenvalue of $\mathbf{A}$ closest to σ.

Note, that a naive application of Algorithm 1 would to compute

$$\mathbf{y}^{(k)} = (\mathbf{A} - \sigma\mathbf{I})^{-1}\mathbf{x}^{(k)},$$

which is numerically unstable due to the computation of the matrix inverse. To avoid this computation we obtain $\mathbf{y}^{(k)}$ by solving a linear system. We arrive at the following algorithm, where the changes compared to Algorithm 1 are marked in red.

Algorithm 2: Inverse iterations

Given

- a diagonalisable matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$,
- a shift $\sigma \in \mathbb{R}$, such that $\mathbf{A} - \sigma \mathbf{I}$ is invertible
- an initial guess $\mathbf{x}^{(1)} \in \mathbb{R}^n$

we iterate for $k = 1, 2, \ldots$:

1. **Solve $(\mathbf{A} - \sigma \mathbf{I})\mathbf{y}^{(k)} = \mathbf{x}^{(k)}$ for $\mathbf{y}^{(k)}$** .
2. Find the index m such that $y_m^{(k)} = \|\mathbf{y}^{(k)}\|$
3. Set $\alpha^{(k)} = \frac{1}{y_m^{(k)}}$ and $\beta^{(k)} = \sigma + \frac{x_m^{(k)}}{y_m^{(k)}}$.
4. Set $\mathbf{x}^{(k+1)} = \alpha^{(k)} \mathbf{y}^{(k)}$

Note the additional change in step 3: In Algorithm 1 we obtained the estimate of the dominant eigenvalue as $y_m^{(k)} / x_m^{(k)}$. Here this estimate approximates $\frac{1}{\lambda_1 - \sigma}$, the dominant eigenvalue of $(\mathbf{A} - \sigma \mathbf{I})^{-1}$. Therefore an estimate of λ_1 itself is obtained by solving

$$\frac{y_m^{(k)}}{x_m^{(k)}} = \frac{1}{\lambda_1 - \sigma} \quad \Longrightarrow \quad \lambda_1 = \sigma + \frac{x_m^{(k)}}{y_m^{(k)}}$$

for λ_1, which yields exactly the expression shown in step 3 of Algorithm 2.

Before we take a look at the implementation, one final point. Each iteration of Algorithm 2 requires the solution of a linear system with the same system matrix $\mathbf{B} = \mathbf{A} - \sigma \mathbf{I}$. Here, we will attempt this using direct methods, that is PLU factorisation. Since $\mathbf{B}$ does not change between iterations, the factorisation itself only needs to be computed once, which we do right before entering the loop. Since for dense matrices computing the factorisation scales $O(n^3)$, but solving linear systems based on the factorisation only scales $O(n^2)$ (recall chapter 6), this reduces the cost per iteration.

```
inverse_iterations (generic function with 1 method)
```

```julia
function inverse_iterations(A, σ, x; maxiter=100)
    # A:  Matrix
    # σ:  shift
    # x:  initial guess

    n = size(A, 1)
    x = normalize(x, Inf)

    fact = lu(A - σ*I)   # Compute LU factorisation of A-σI

    history = Float64[]
    for k in 1:maxiter
        y = fact \ x
        m = argmax(abs.(y))
        α = 1 / y[m]
        β = σ + x[m] / y[m]
        push!(history, β)
        x = α * y
    end

    (; x, λ=last(history), history)
end
```

Inserting $\mathbf{A} - \sigma\mathbf{I}$ in the place of $\mathbf{A}$ in (6) and recalling the eigenvalue ordering (8), i.e. $|\lambda_n - \sigma| \geq |\lambda_{n-1} - \sigma| \geq \cdots |\lambda_2 - \sigma| > |\lambda_1 - \sigma| > 0$ one can show similarly that inverse iterations converge again linearly with rate

$$r_{\text{inviter}} = \lim_{k\to\infty} \frac{|\beta^{(k+1)} - \lambda_1|}{|\beta^{(k)} - \lambda_1|} = \left|\frac{\lambda_1 - \sigma}{\lambda_2 - \sigma}\right| \qquad \text{as } k \to \infty. \tag{9}$$

This implies that convergence is fastest if σ is closest to the targeted eigenvalue λ_1 than to all other eigenvalues of $\mathbf{A}$.

To demonstrate this rate we again consider the case of diagonalising a triangular matrix

```
5×5 Matrix{Float64}:
 1.0  1.0   1.0   1.0  1.0
 0.0  0.75  1.0   1.0  1.0
 0.0  0.0   0.6   1.0  1.0
 0.0  0.0   0.0  -0.4  1.0
 0.0  0.0   0.0   0.0  0.0
```

```julia
begin
    λT = [1, 0.75, 0.6, -0.4, 0]   # The reference eigenvalues we will use
    T  = triu(ones(5, 5), 1) + diagm(λT)
end
```

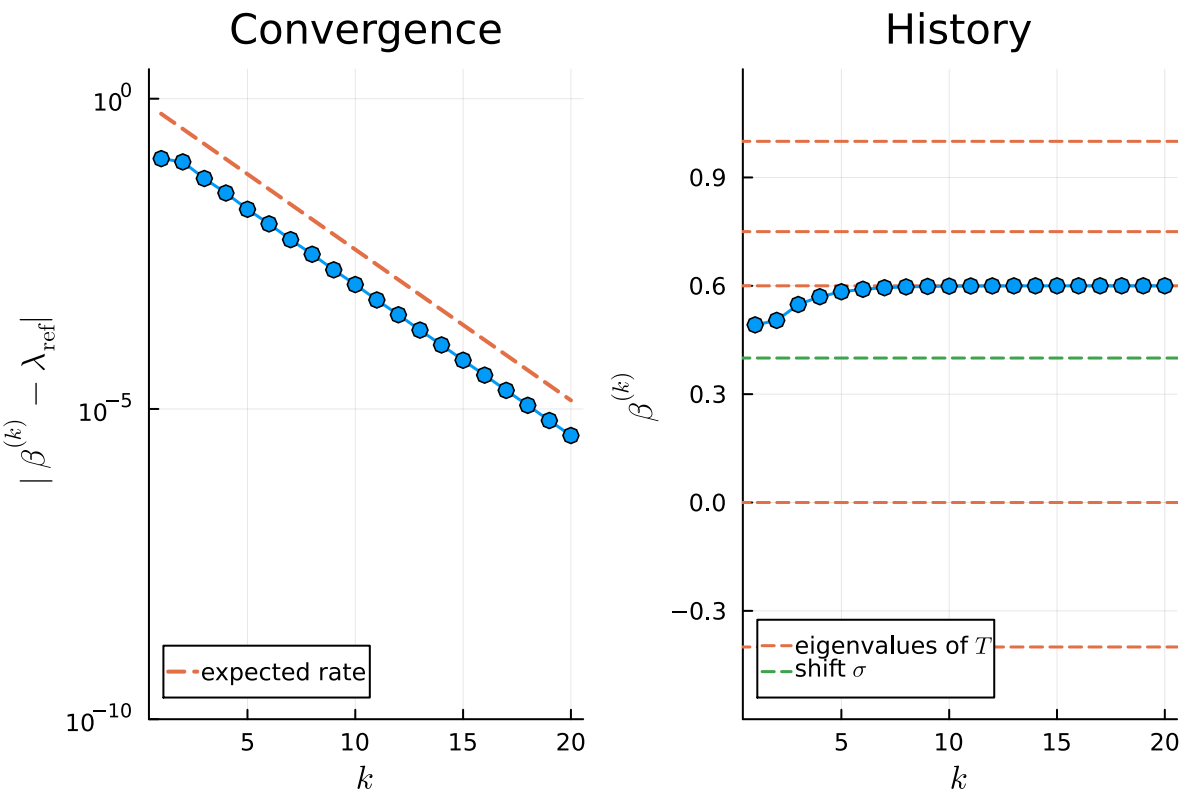
This time the slider allows to tune the value of $\boldsymbol{\sigma}$ (which we store in the variable `σT`):

- `σT =` 0.4

This value of $\boldsymbol{\sigma}$ results in a rate:

```
0.5714285714285713
```

```
begin
    # compute |λ_5 - σ| > |λ_4 - σ| > |λ_3 - σ| > |λ_2 - σ| > |λ_1 - σ|
    # We need sort to get the data in that order
    diff_sorted = sort(abs.(λT .- σT); rev=true)

    # take last element (|λ_1 - σ|) and last-but-one (|λ_2 - σ|)
    r_inviter = diff_sorted[end] / diff_sorted[end-1]
end
```



⚠ TODO ⚠

Give an example using a 3x3 tridiagonal matrix using power iterations, inverse power iterationsn ($\sigma = 0$), inverse power iterations with non-zero shift. One case should not be converging due to two dominant eigenvalues. In each case indicate the eigenvalue to which one converges and the rate of convergence.

```
TODO(md"Give an example using a 3x3 tridiagonal matrix using power iterations,
inverse power iterationsn ($σ=0$), inverse power iterations with non-zero shift.
One case should not be converging due to two dominant eigenvalues. In each case
indicate the eigenvalue to which one converges and the rate of convergence.")
```

Optional: Dynamic shifting

In the discussion in the previous section we noted that the convergence of inverse iterations is best if σ is chosen close to the eigenvalue of the matrix $\mathbf{A}$. Since inverse iterations (Algorithm 2) actually produce an estimate for the eigenvalue in each iteration (the $\beta^{(k)}$), a natural idea is to *update* the shift.

In other words instead of using the same σ in each iteration of Algorithm 2, we dynamically update $\sigma = \beta^{(k)}$ once the new eigenvalue estimate $\beta^{(k)}$ has been computed. This also implies that for solving the linear system in step 1, i.e.

$$(\mathbf{A} - \sigma \mathbf{I})\, \mathbf{y}^{(k)} = \mathbf{x}^{(k)}$$

the system matrix is different for each k, such that pre-computing the PLU factorisation once before entering the iteration loop is no longer possible.

We arrive at the following implementation for an **inverse iteration algorithm with dynamic shifting**:

```
dynamic_shifting (generic function with 1 method)
```

```julia
function dynamic_shifting(A, σ, x; maxiter=100, tol=1e-8)
    # A:  Matrix
    # σ:  shift
    # x:  initial guess

    n = size(A, 1)
    x = normalize(x, Inf)

    history = Float64[]
    for k in 1:maxiter
        y = (A - σ*I) \ x   # PLU-factorise (A - σ*I) and solve system
        m = argmax(abs.(y))
        α = 1 / y[m]
        β = σ + x[m] / y[m]
        push!(history, β)
        x = α * y

        if abs(σ - β) < tol  # We are converged, so exit the iterations.
            break
        end
        σ = β
    end

    (; x, λ=last(history), history)
end
```

A consequence of the fact that we need to compute the PLU factorisation in each iteration is that the overall cost of `dynamic_shifting` is larger than the cost of `inverse_iterations`, but the convergence is also improved: We go from linear to **quadratic convergence** (see chapter 8.3 of Driscoll, Brown: Fundamentals of Numerical Computation).

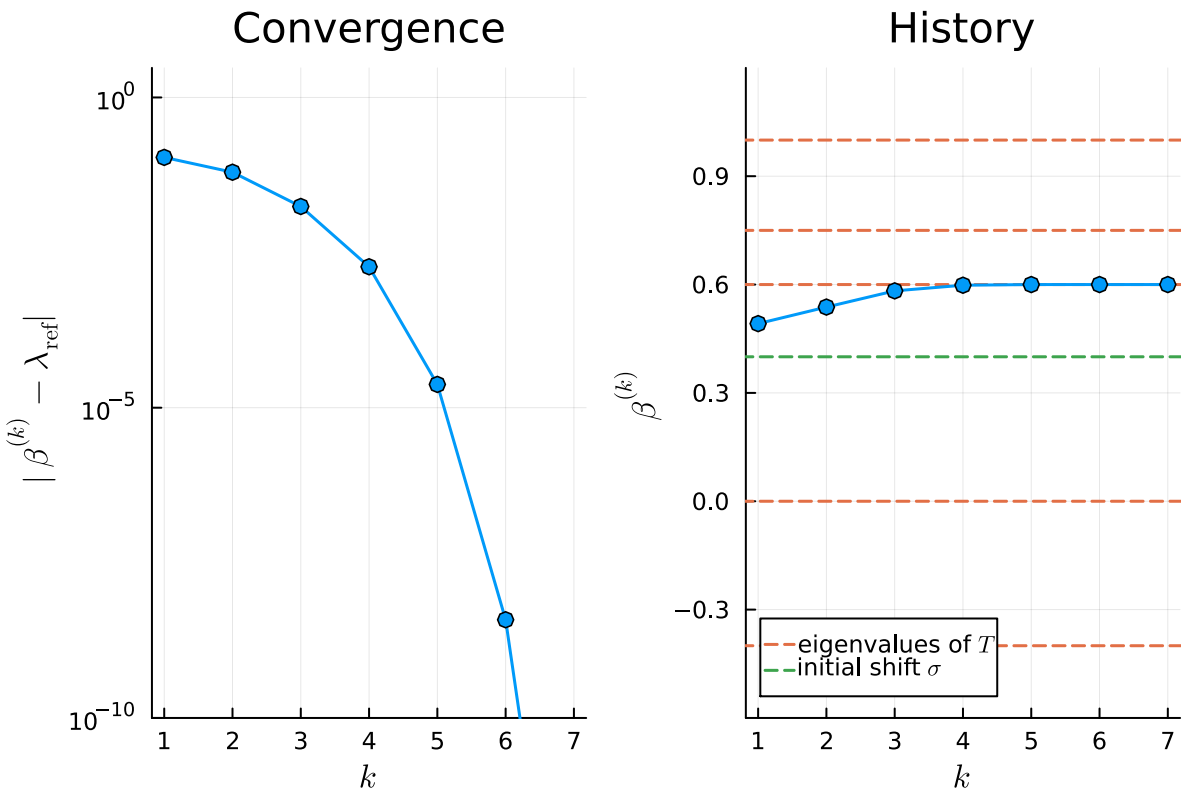
This is easily checked numerically. We use the same matrix

```
5×5 Matrix{Float64}:
 1.0  1.0   1.0   1.0  1.0
 0.0  0.75  1.0   1.0  1.0
 0.0  0.0   0.6   1.0  1.0
 0.0  0.0   0.0  -0.4  1.0
 0.0  0.0   0.0   0.0  0.0
```

```julia
T
```

and introduce another slider to tune the value of the *initial* shift σ, which we store in the variable `σD`:

- `σD =` 0.4



The much faster quadratic convergence is clearly visible.

```
xstart = randn(size(T, 2));
```

Numerical analysis

1. Introduction
2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Interpolation
6. Direct methods for linear systems
7. Iterative methods for linear systems
8. Eigenvalue problems
9. Numerical integration
10. Numerical differentiation
11. Initial value problems
12. Boundary value problems